DEBEZIUM COMMUNITY SHOWCASE · EPISODE 1

A Verified Theoretical Basis for Incremental Snapshots in Debezium

Andreas Andreakis

Created DBLog at Netflix

aandreakis.com

WHY THIS TALK

Why this talk exists

THE QUESTION

Are incremental snapshots sound?

Original design built at Netflix. Now adopted in open source, including Debezium and Flink CDC.

PAST STRUGGLE

Theory, proofs and formal verification.

Rare skills, out of reach for practitioners.

WHAT CHANGED

LLMs write and check the proofs.

A theoretical study is now within a practitioner's reach.

This talk presents the result: how incremental snapshots work under the hood, what they guarantee and how the theory was built.

Not a usage guide for incremental snapshots in Debezium. Use the official documentation for this.

OUTLINE

The talk in three parts

1 Origins

How DBLog came to be, the problem it solves and how it works.

2 Theory

What the theoretical contract is and what it provides (presented practitioner friendly).

3 Formal verification

How it was done and what was learned.

I left Netflix in summer 2021. Everything about Netflix in this talk comes from its public engineering posts and papers, cited on the slides.

PART 1 OF 3

Origins

How DBLog came to be, the problem it solves and how it works.

Incremental snapshots in Debezium

HOW DBLOG CAME TO BE

Teams building for the Netflix Studio

Shot scheduling

what is filmed when, and where

Contracts

deals with talent and vendors

Subtitles

localization into many languages

Many more ...

talent, VFX, dubbing, asset management, finance

Several internal teams build products for the studio. Each team owns its datastores.

Primaries mostly MySQL or PostgreSQL, RDS or Aurora on AWS, and Cassandra. Secondary stores such as search (Elasticsearch) or data warehouses (Iceberg) sit next to them and have to follow the primary.

A typical heterogeneous setup.

Source: Netflix Technology Blog, “How Netflix Scales its API with GraphQL Federation (Part 1)”, 2020, “Delta: A Data Synchronization and Enrichment Platform”, 2019, and “Data Movement in Netflix Studio via Data Mesh”, 2021

HOW DBLOG CAME TO BE

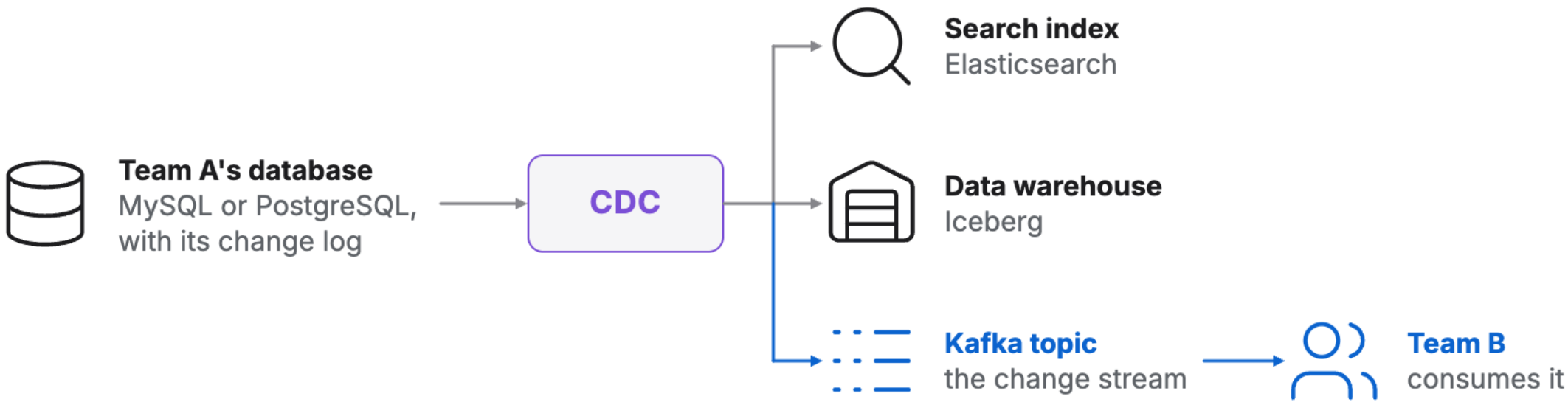
Two needs, one change stream

Keep secondary stores in sync

Search indexes, warehouses and caches must follow the primary database.

Share data across teams

Other teams consume the changes continuously as a stream via Kafka topics.

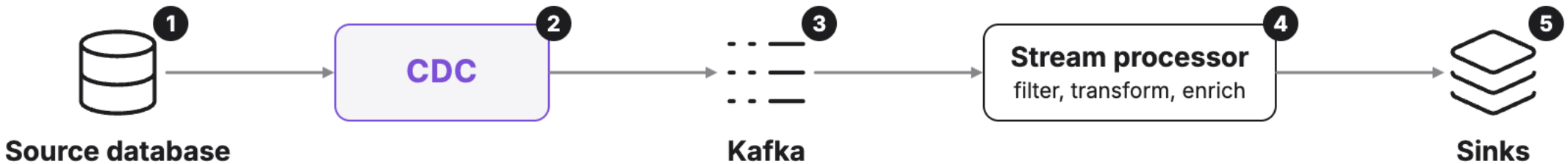


Source: Netflix Technology Blog, “Delta: A Data Synchronization and Enrichment Platform”, 2019, and “Data Movement in Netflix Studio via Data Mesh”, 2021

HOW DBLOG CAME TO BE

Beyond CDC: Full state, surgical fixes, periodic repairs

A new sink needs the existing rows. And not only once. Multiple times during pipeline setup (aka dev phase) and for fixing issues in prod, surgical or wider.



1 Source database

A config change disables CDC. Replication slot was dropped.

2 CDC

The source log expired before the connector caught up. A type conversion bug creates a wrong value.

3 Kafka

Events expire before a slow consumer reads them. A late reader misses deletes that compaction already removed.

4 Processing

A step dropped rows. A lookup joined stale data or a join lost its state.

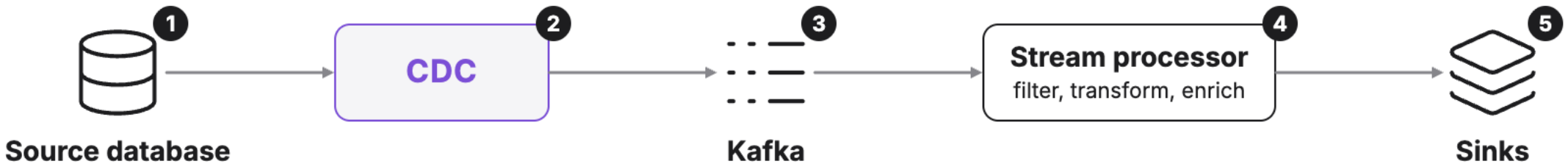
5 Sinks

A new sink starts empty. An older write lands after a newer one.

HOW DBLOG CAME TO BE

Beyond CDC: Full state, surgical fixes, periodic repairs

A new sink needs the existing rows. And not only once. Multiple times during pipeline setup (aka dev phase) and for fixing issues in prod, surgical or wider.



1 Source database

A config change disables CDC. Replication slot was dropped.

2 CDC

The source log expired before the connector caught up. A type conversion bug creates a wrong value.

3 Kafka

Events expire before a slow consumer reads them. A late reader misses deletes that compaction already removed.

4 Processing

A step dropped rows. A lookup joined stale data or a join lost its state.

5 Sinks

A new sink starts empty. An older write lands after a newer one.

Requirements: (re)trigger a copy from the source, for a whole table or specific keys, while the stream keeps running. No new pipeline, no cutover, no stop of the change stream.

Periodic repairs are a best practice too, as you may not know something is off. Ideally without interrupting the flow of events.

HOW DBLOG CAME TO BE

Coordinating CDC events with (full) copies is tricky

Re-triggering a copy while the stream runs needs coordination.

The history must stay correct: the newest value wins and deleted rows do not reappear.

Classic approaches

Stop traffic, dump, note the log position, resume.

Snapshot, then the log from its position. Lock-free only with vendor-specific support, like PostgreSQL replication slots. Otherwise a lock pauses writes, at least briefly, as in Debezium for MySQL.

Pause the log, select, resume. The sink time-travels until the log catches up.

Copy tables. Insert the rows in chunks into a scratch copy table, so they arrive through the log. Adds write pressure on the source, needs locks.

What we wanted

The change event flow is not interrupted.

Copies at any time, for a whole table or selected keys.

Pause and resume on big tables.

Minimal source impact and no locks.

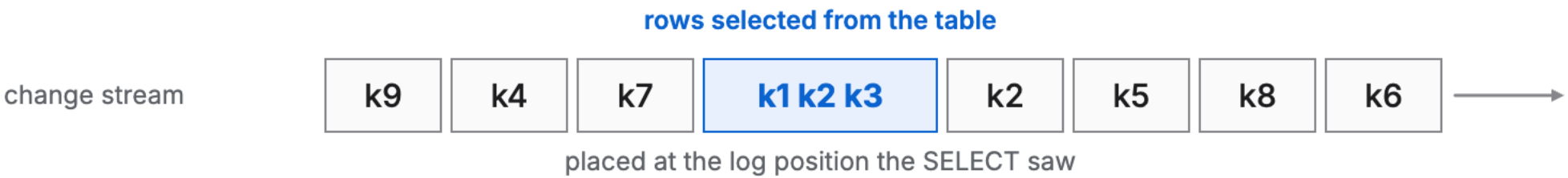
Nothing vendor-specific, so one implementation can apply on several databases.

A fast first load is not the goal of this design.
Using a high throughput bootstrap first (like snapshot based) and DBLog after combines well.

DBLOG

The idea: inject SELECT results into the change stream

What if the Connector injects **additional events** alongside the CDC events that it receives from the source: rows selected from the current table state, placed so that the change history stays correct.



Convenient if the database tells you the position

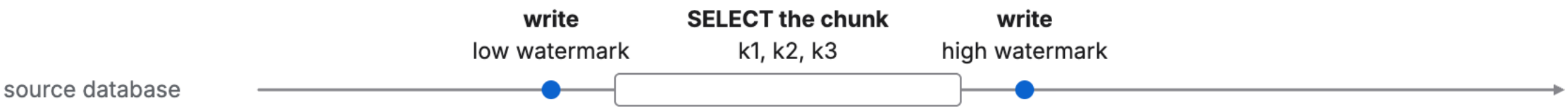
MariaDB can report the log position of a consistent snapshot read. The selected rows get this position assigned.

Many databases do not

So the question was: is there one approach for a wide range of databases? Relational systems share a linear log of committed changes, keyed rows and SELECTs that return the latest committed state.

DBLOG · STEP 1 OF 4

The solution: SELECT between two watermarks

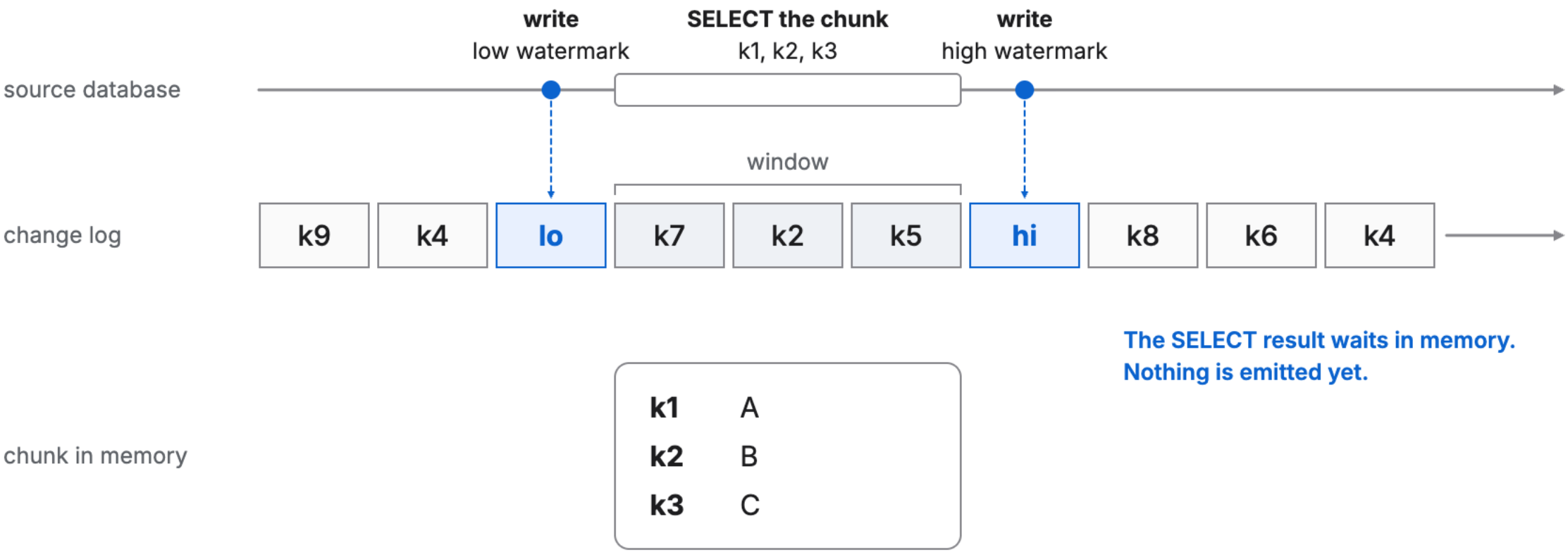


chunk in memory

k1	A
k2	B
k3	C

DBLOG · STEP 2 OF 4

The solution: SELECT between two watermarks

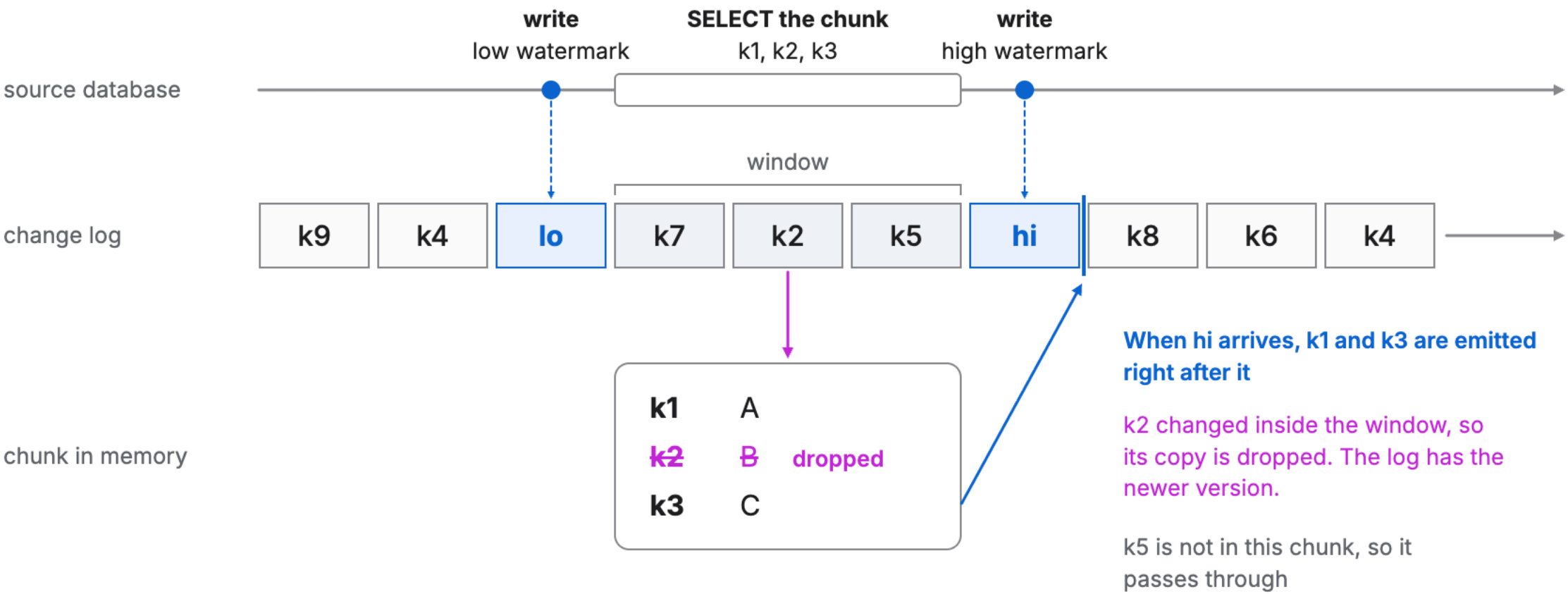


The SELECT result waits in memory.
Nothing is emitted yet.

The SELECT ran at some unknown point within the watermark window.
We then observe the committed watermarks in the change log and the keys that changed within the window.

DBLOG · STEP 3 OF 4

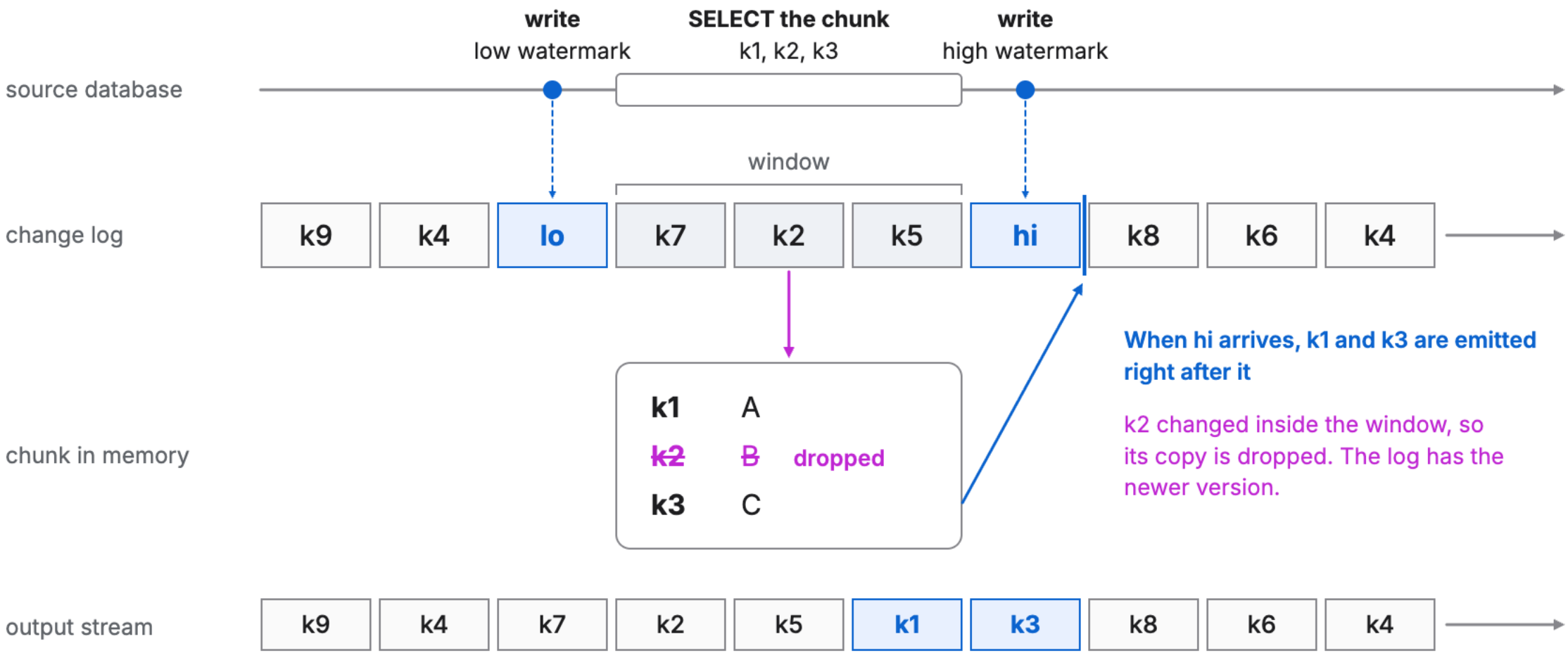
The solution: SELECT between two watermarks



We drop selected keys that changed in the window because we do not know if the SELECT happened before, at, or after the changes. The log contains the latest state, so it is safe to drop from the SELECT.

DBLOG · STEP 4 OF 4

The solution: SELECT between two watermarks



The table's events in log order, with the copied rows **k1** and **k3** right after hi. k2 is not copied, the log already has it.

A whole table, chunk by chunk, or specific keys

the table, rows ordered by primary key

pk	column 2	column 3	
k1			chunk 1
k2			
k3			
k4			chunk 2
k5			
k6			
k7			chunk 3
k8			
k9			

... and so on, in primary key order, until the max key — chunk 4 ...

Whole table

Chunks walk the table in primary key order. Each SELECT starts after the last key read, WHERE pk > last ORDER BY pk LIMIT n, until the max key is reached, taken when the copy started.

Specific keys

For surgical fixes a specific set of keys can be specified, WHERE pk IN (...) and executed as one chunk or more.

Either way, every chunk runs the same algorithm: Its own watermark window and the log wins. Between chunks the log flows as usual.

DBLOG

Writing to the source: an accepted practice*

A CDC tool that writes to the source can seem unattractive at first. But small, controlled writes to a dedicated table are already an established observability technique.

Debezium heartbeats

A configured query can write to a heartbeat table when captured data is quiet, so the connector's position keeps moving.

Oracle GoldenGate

Updates heartbeat tables on the source and carries those records through replication to measure end-to-end lag.

DBLog watermarks

Occasional updates of a single-row table.

A deployment that accepts heartbeats should be fine with watermarks.

*Not every team likes writes to the source. Debezium's read-only modes exist for them.

DBLOG

Why this is relevant for the Debezium community

Debezium's incremental snapshots follow the DBLog design.

In short: chunked selects that are bracketed within a window and log events win on overlap.

MORE ABOUT DBLOG HERE:

NETFLIX BLOG · 2019

DBLog: A Generic Change-Data-Capture Framework

The original post.

netflixtechblog.com/dblog-a-generic-change-data-capture-framework-69351fb9099b

ARXIV · 2020

DBLog: A Watermark Based Change-Data-Capture Framework

The paper with more details.

arxiv.org/abs/2010.12597

GITHUB · 2026

dblog-impl: a reference implementation

Not an official Netflix implementation. Created by me post-Netflix based on the paper.

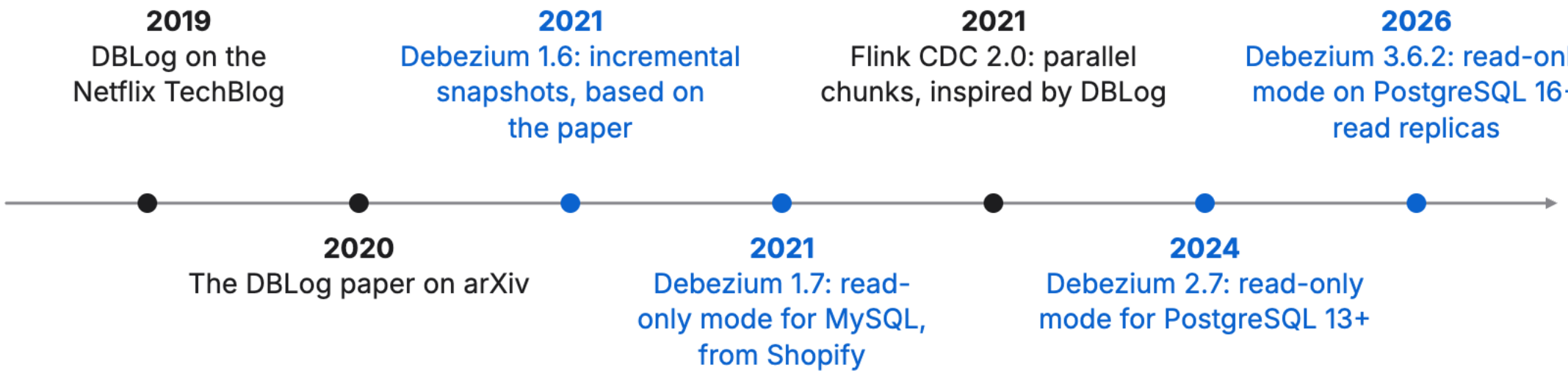
For educational purposes only, not maintained, not for production.

Use a well maintained CDC project, like Debezium, for production.

github.com/aandreakis/dblog-impl

DBLOG

From DBLog to Debezium and Flink CDC



Open source made the idea travel and added new variants: read-only modes from the Debezium team and Shopify, and parallel chunks from Flink CDC.

PART 2 OF 3

Theory

What the contract is and what it provides, kept practitioner friendly.

DEBEZIUM

Incremental snapshots in Debezium

THREE WAYS TO SNAPSHOT

Initial snapshot
When the connector first starts. Copies all tables first, change events after. A failure starts it over.

Blocking snapshot
On demand, for the tables you want. Same kind of copy as the initial. Streaming pauses, then resumes.

Incremental snapshot
Streaming continues. Any time, any captured table with a primary key, or only rows that match a filter. Resumes where it stopped.

START ONE WITH A SIGNAL

```
INSERT INTO inventory.debezium_signal (id, type, data)
VALUES ('ad-hoc-1', 'execute-snapshot',
  '{"data-collections": ["inventory.orders"],
    "type": "incremental"}');
```

The signal table is named in signal.data.collection. Signals can also arrive through Kafka, JMX or a file, but the table still carries the watermarks.

Other signals pause, resume or stop a running snapshot. With additional-conditions, only matching rows are copied.

Notifications report the progress.

As of Debezium 3.7, in most connectors, including MySQL, MariaDB, PostgreSQL, SQL Server, Oracle, Db2, Informix, MongoDB, CockroachDB.

DEBEZIUM

Debezium marks the window in three ways

Signal table (default)



An open marker and a close marker in the signal table around the SELECT, read back from the log.

Read-only: MySQL, MariaDB



A GTID position, sampled before and after the SELECT: the executed GTID set on MySQL, GTID_BINLOG_POS on MariaDB.

Read-only: PostgreSQL



pg_current_snapshot(), sampled before and after the SELECT.

In all three modes, a row that changed inside the window is dropped from the chunk.
The other rows are emitted when the window closes.

Since 1.6 (2021). Read-only: MySQL since 1.7 (Shopify), PostgreSQL 13+ since 2.7 (2024), PostgreSQL 16+ read replicas since 3.6.2 (2026).

ONE CONTRACT: GENERALIZED DBLOG

Different mechanics, one contract

CONDITIONS ON THE SOURCE DATABASE

A commit-ordered log
Every committed change appears once, in commit order.

Full row images
Each change event carries the whole row.

Committed, complete reads
A chunk SELECT returns every committed row in its range.

CONDITIONS ON THE CONNECTOR

	Chunks by key	A window around each read	In the window the log wins	Release after the window
DBLog	by key	watermark writes	drop the copy	at HIGH
Debezium signal table	by key	open and close markers	drop the copy	at close
Debezium read-only	by key	GTID or snapshot samples	drop the copy	when the window closes

Together they form one umbrella, Generalized DBLog. Whatever meets both sets of conditions is an instance of it.

ONE CONTRACT: GENERALIZED DBLOG

Even parallel chunks fit the same contract

CONDITIONS ON THE CONNECTOR

	Chunks by key	A window around each read	In the window the log wins	Release after the window
DBLog	by key	watermark writes	drop the copy	at HIGH
Debezium signal table	by key	open and close markers	drop the copy	at close
Debezium read-only	by key	GTID or snapshot samples	drop the copy	when the window closes
Flink CDC	**by key, in parallel**	**sampled log offsets**	**merge into the copy**	**at HIGH, then skip older events**

Chunk workers run in parallel. Each one samples LOW, reads its chunk, samples HIGH, and merges the window's log events into the chunk. After the last chunk, one log reader streams on and skips what the chunks already processed.

THE RESULT: MIX AND MATCH

One contract, interchangeable parts

Whatever meets the contract can be combined.
The proof does not care where a part came from.

Parallel chunks in Debezium (as a suggestion)

Flink CDC's per-chunk windows meet the rules and so do Debezium's markers and read-only samples. A Debezium that reads chunks in parallel would still be covered.

Room for what comes next

A new mechanism only has to show that it meets the four rules and the source conditions. Then the proof covers it and every other implementation can apply it.

The proof is about the rules, not about a codebase. Any implementation that follows them is covered.

THE RESULT: SNAPSHOT EQUIVALENCE

What the contract gives: Snapshot equivalence

Follow the contract and your sink converges to the latest state of the source.

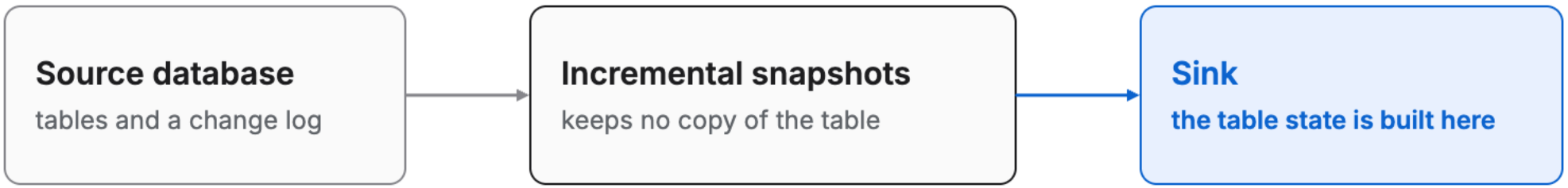
A snapshot is a copy of the whole table at one instant. Incremental snapshots never take one. Yet, your sink ends up exactly as if one had been taken.

Intuition: the snapshot is built key by key, from the log events plus the injected copied rows (windowing guarantees that they preserve the history correctly).

And it keeps moving: every later change event updates the snapshot in place at the sink. That is what makes it **incremental**.

THE RESULT: SNAPSHOT EQUIVALENCE

The state is built downstream



The connector emits one stream of copied rows and change events.

Your sink applies this stream and builds the table state from it.

The connector keeps no copy of the table. The state exists only where the stream is applied: a table, an index, a compacted topic, etc.

THE RESULT: SNAPSHOT EQUIVALENCE

When is my sink in-sync?

WHILE IT RUNS

Rows of finished chunks match the source. Rows not copied yet may be missing, or stale if your sink already had them.

→

LAST CHUNK DONE

Every copied row matches the source at one point in the log.

→

FROM THEN ON

The point moves forward with each new change: the sink converges to the latest state of the source.

Wait until the last chunk has completed if you need all pre-existing keys covered.

If the source keeps changing, the sink may be lagging behind though it keeps on advancing forward.
This is an implication of CDC itself.

PART 3 OF 3

Formal verification

Formal verification with LLMs.
How it was done and what was learned.

Incremental snapshots in Debezium

FORMAL VERIFICATION WITH LLMS

Why this was out of reach pre-LLMs

Two rare skills

Distributed systems theory and proof engineering, in one head, or in two groups that have to understand each other.

Slow

Formal verification takes time and waiting for an expert takes time too. Every change to the theory means proving again.

Scarce and expensive

Such experts are scarce and may have no time or interest to review your work. If you can hire one, it is expensive.

For practitioners outside a research group, formal verification was infeasible. Recent LLMs finally made this possible.

FORMAL VERIFICATION WITH LLMS

What was verified

**The proofs cover the theory and a step-by-step description of each mechanism.
Debezium and Flink CDC, as documented, follow those steps.
No Java code is verified, no running connector is certified.**

Every variant satisfies the contract

Classic DBLog, Debezium's signal-table mode and its MySQL and PostgreSQL read-only modes, and Flink CDC.

The contract gives snapshot equivalence

Replaying the emitted stream equals the source at one point in the log, for every key in the snapshot.

Assumes: a log in commit order with full row images, on PostgreSQL via REPLICA IDENTITY FULL or an equivalent, chunk reads that skip no row, rows with keys and a sink that applies the stream in order, latest value per key.

FORMAL VERIFICATION WITH LLMS

Three tools, three kinds of evidence

Isabelle/HOL

An interactive proof assistant from the mid-1980s. Every theorem must pass through a small trusted kernel.

Here: the complete theory, about 5,000 lines.

Lean 4

A newer proof assistant and programming language with a large mathematics library.

Here: the core theorem proved a second time, in about 2,300 lines, by a second independent checker.

TLA+

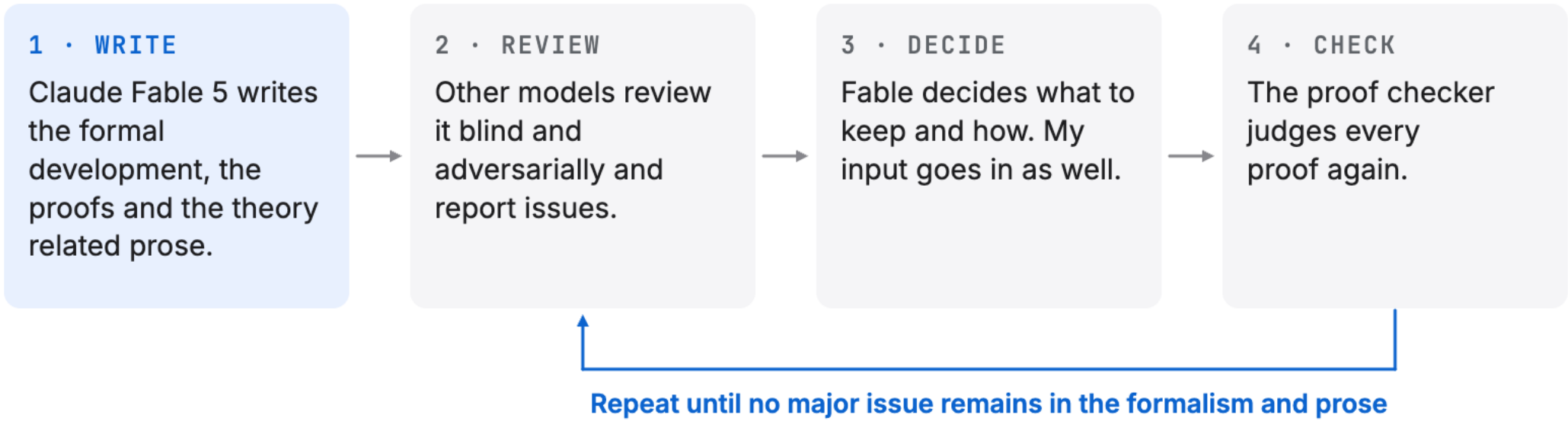
Leslie Lamport's language for specifying concurrent systems. Its model checker TLC tries every interleaving of a small setup.

Here: five models of the protocols, 25 runs, up to 74 million states. Evidence, not proof.

Three kinds of evidence because each tool can have its own blind spots.

FORMAL VERIFICATION WITH LLMS

How it was done: one writer, a review loop



Reviewers: Fable 5 in separate sessions, Claude Opus 5, ChatGPT 5.6 Sol and GPT 6 Astra

Most rounds went into the theory related paper text, not the proofs. Readability needed explicit attention: text from recent models has become harder to read.

The checker guarantees that every proof is correct and anyone can re-run it. It cannot guarantee that the theorems say the right thing about the real systems and how rigorous the LLM reviews really are is hard to assess.

FORMAL VERIFICATION WITH LLMS

How the theory was built: two papers, May to September

MAY 2026	**A Theoretical Study of DBLog: Certified Virtual Cuts for a Snapshot-Equivalent Replay of Live Databases** Isabelle/HOL theory of classic DBLog and its snapshot equivalence. First theory file 7 May, on arXiv 29 May, rebuilt without unproved assumptions for the Archive of Formal Proofs by 12 June. 38 theory files, 16,600 lines. Written with Claude Opus 4.7 and ChatGPT 5.5.	Isabelle: 3.6 min
SEP 2026	**Generalized DBLog: A Verified Contract for Interleaving Copied Rows with a Change Log** The theory of the four rules, with the classic DBLog of the May paper plugged in as a worked instance. Isabelle core of 9 theory files and about 4,600 lines in 37 hours, 5,000 today, then five TLA+ models and the Lean port. On arXiv 8 September.	Isabelle: 4 min Lean: 3 sec TLC: 12 min

LEARNED

A green build is not the same as a correct model. The checker said everything was proved and an independent review still found mistakes in what had been modeled. Each one is now a regression check that runs with every build.

The main result, snapshot equivalence, was the cheap part. Most of the work went into the plumbing: how chunks are buffered and merged, and into examples that show each rule is really needed.

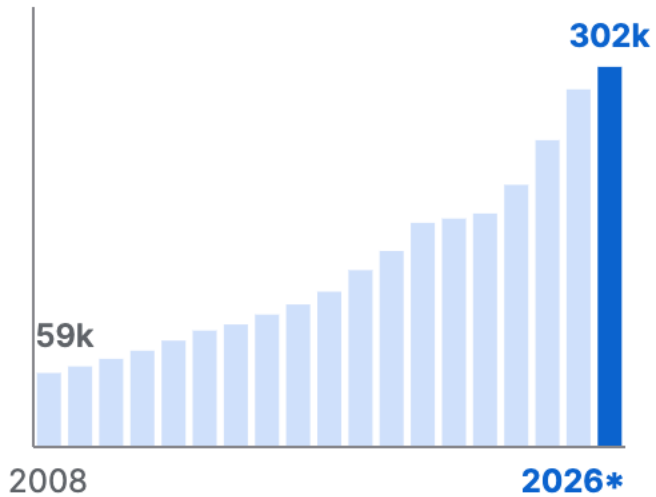
Re-checking everything takes minutes, like a test suite. So every change was re-proved the same day, across some 700 commits on these two papers.

All checks ran on Apple M2 Max.

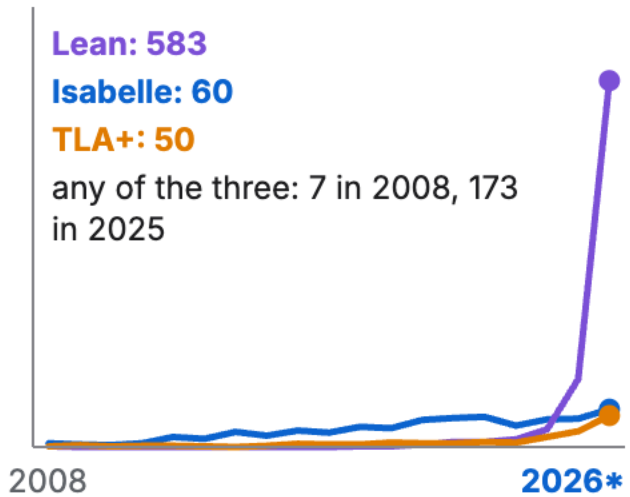
FORMAL VERIFICATION WITH LLMS

Who writes and reads proofs now: agents, much more likely than you

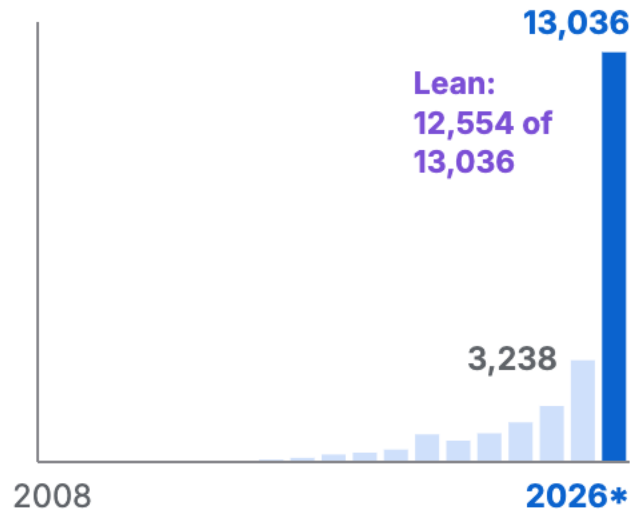
arXiv papers per year



Abstracts mentioning formal proofs



GitHub repos with formal proofs



Formal content is growing fast. The proofs are written by and for machines.

* 2026 to October 25. Sources: arxiv.org/stats, arXiv API counts of abstracts mentioning Isabelle, Lean 4, mathlib or TLA (text matching, approximate: arXiv search drops the plus in TLA+), GitHub repository search by primary language (Isabelle, Lean, TLA). GitHub's Lean covers Lean 3 and Lean 4.

SUMMARY

DBLog inspired incremental snapshots in Debezium.
Windowed chunks and the log wins.

Every variant satisfies one contract and the contract gives you snapshot equivalence.
Your sink converges to the latest state of the source.

The proofs were written and reviewed by LLMs.
Adding confidence that was infeasible until recently.

Paper	arxiv.org/abs/2609.08160
Code	github.com/aandreakis/generalized-dblog-paper

Thank you. Questions, ideas, working together: reach out.

aandreakis.com · linkedin.com/in/andreas-andreakis · x.com/andreas0x

BACKUP

Backup slides

Delta and Data Mesh at Netflix.
Parallel chunk processing in Flink CDC.
Snapshot equivalence built step by step on two rows.
Re-running the proofs yourself.

Not part of the talk. Kept for questions.

HOW DBLOG CAME TO BE

Delta: one team, many stores

Keep secondary stores in sync

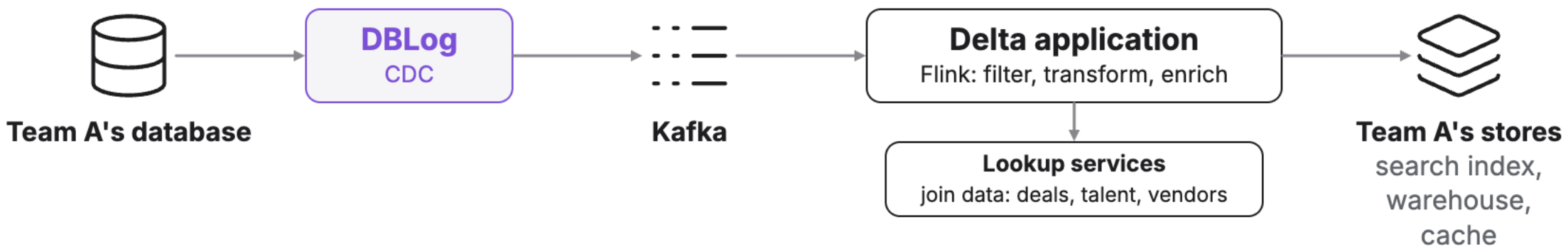
Search indexes, warehouses and caches must follow the primary database.

Share data across teams

Other teams consume the changes continuously as a stream via Kafka topics.

Before, every team wrote its own solution: batch jobs, dual writes, outbox patterns, ...

So we built a managed data movement platform, Delta, with Change-Data-Capture at its core.



Source: Netflix Technology Blog, “Delta: A Data Synchronization and Enrichment Platform” and “DBLog: A Generic Change-Data-Capture Framework”, 2019

HOW DBLOG CAME TO BE

Data Mesh: one stream, many teams

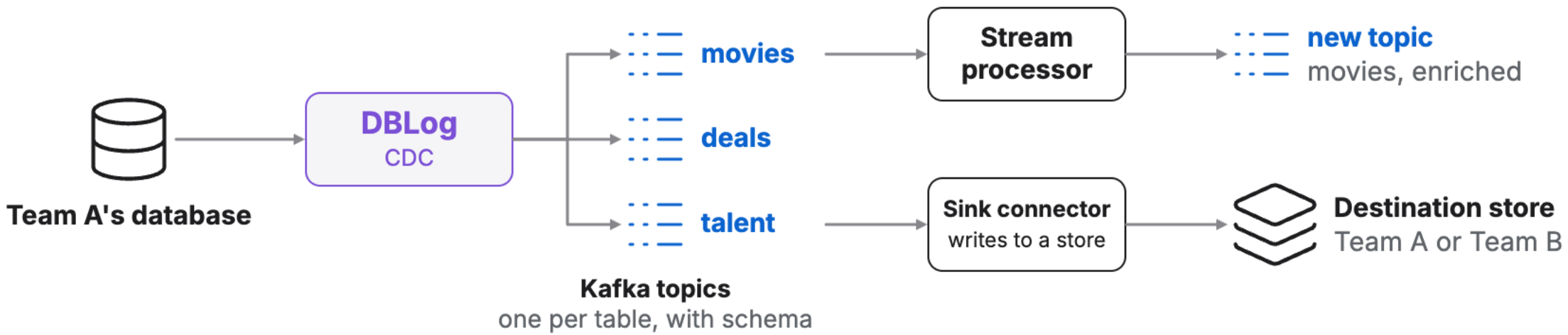
Keep secondary stores in sync

Search indexes, warehouses and caches must follow the primary database.

Share data across teams

Other teams consume the changes continuously as a stream via Kafka topics.

Delta's successor, Data Mesh, gave the Kafka topics unlimited retention, so each topic became a second full copy of its table, always up to date.

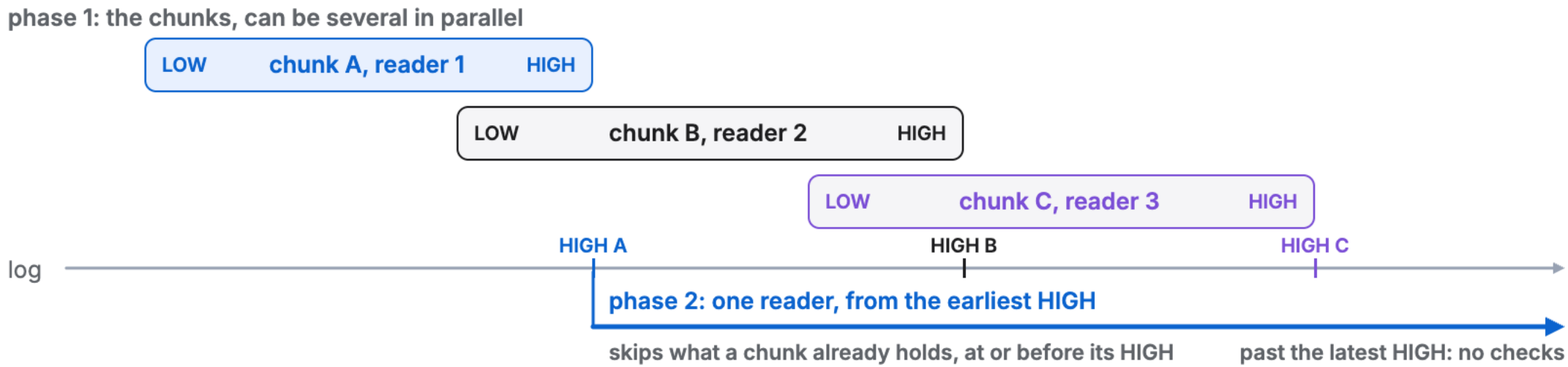


Source: Netflix Technology Blog, “Data Movement in Netflix Studio via Data Mesh”, 2021, and “Data Mesh: A Data Movement and Processing Platform @ Netflix”, 2022

FLINK CDC · BACKUP

Flink CDC: chunks in parallel, then the log

Two phases, one after the other: the log is streamed only once every chunk is done. No writes to the source needed.

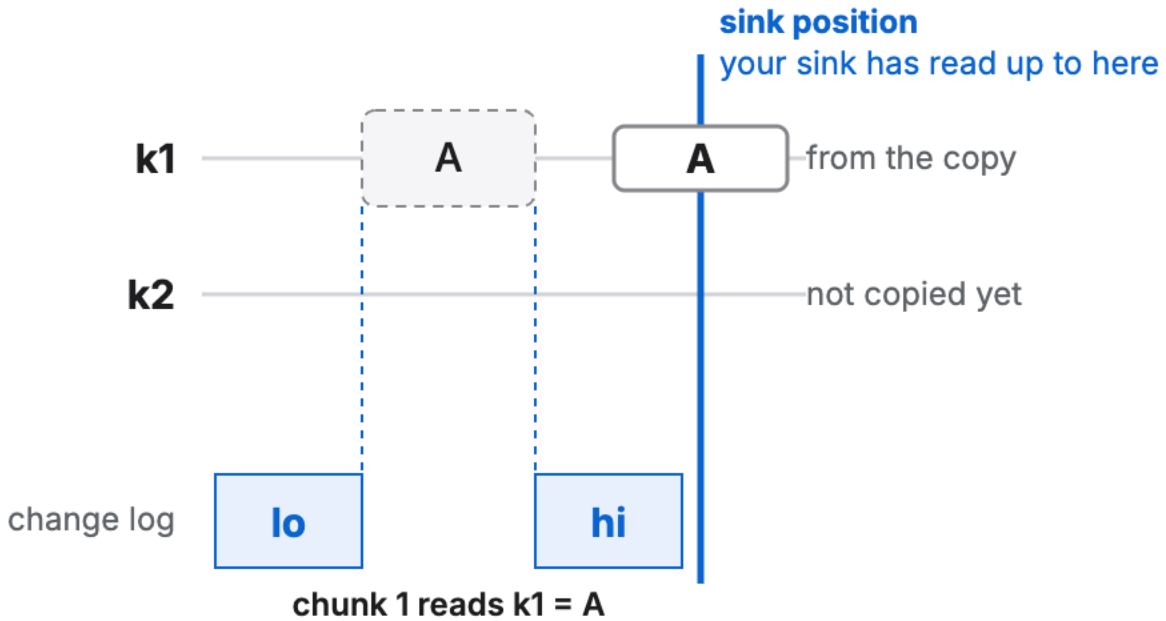


Phase 1: each chunk is read inside its own window, LOW sampled before the SELECT and HIGH after. Changes to its keys inside the window are merged into the copied rows and the resulting chunk is written to the output.

Phase 2: one reader streams the log from the earliest HIGH, so nothing after any window is lost. Events at or before the HIGH of the chunk that owns the key are skipped. There is nothing left to check past the latest HIGH.

SNAPSHOT EQUIVALENCE

Step 1: after the first chunk

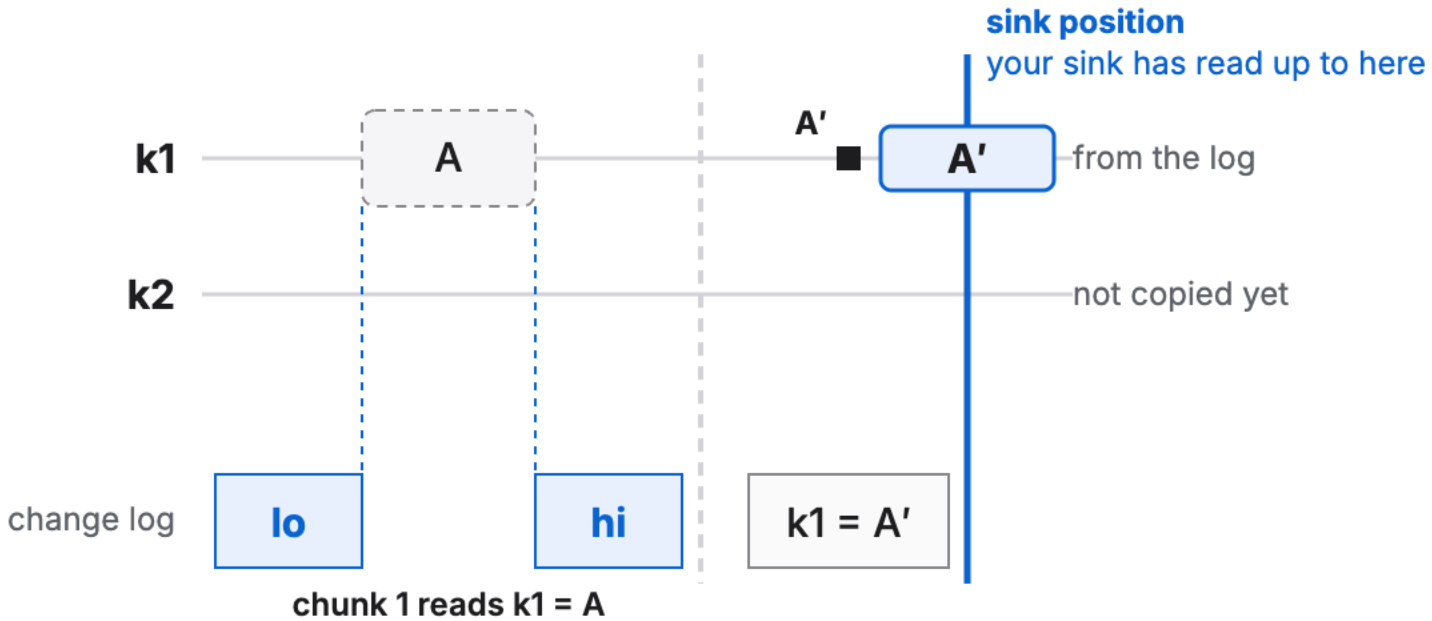


Read down the line: k1 = A, exactly as of this position. k2 is not copied yet, so there is no promise for it.

Chunk 1 read k1 somewhere between lo and hi. Right after hi, its copy is exact, and nothing was locked.

SNAPSHOT EQUIVALENCE

Step 2: after a change event

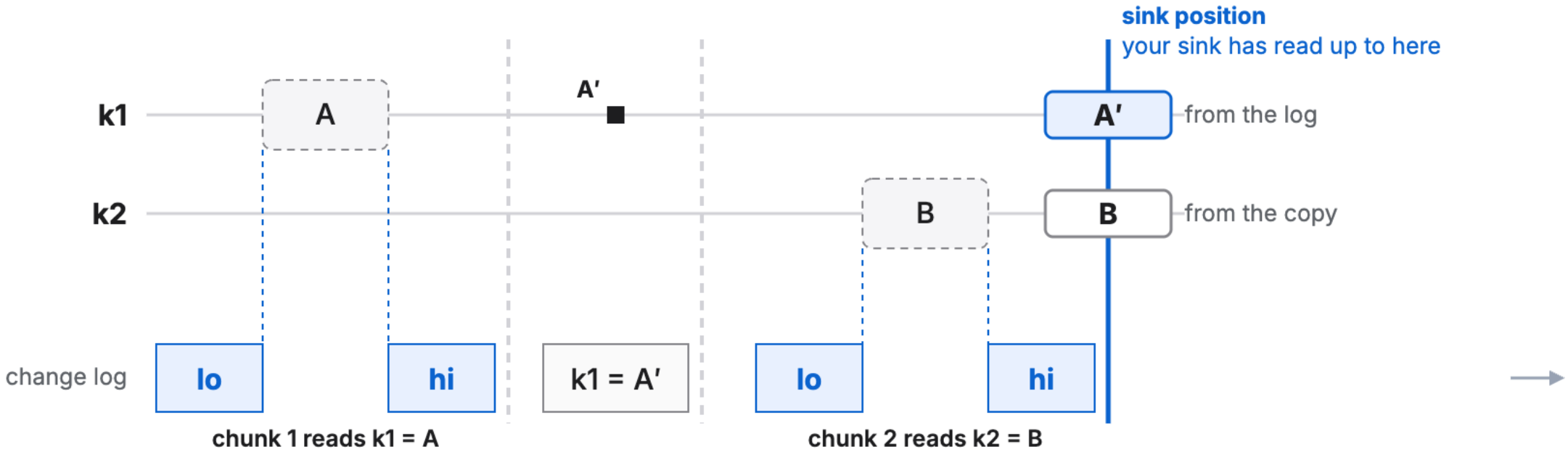


k1 changed to A′. Your sink applies the event, and the line moves: k1 = A′, exactly as of this position.

The copy was exact at its position. Every later change event moves the line forward.

SNAPSHOT EQUIVALENCE

Step 3: after the last chunk

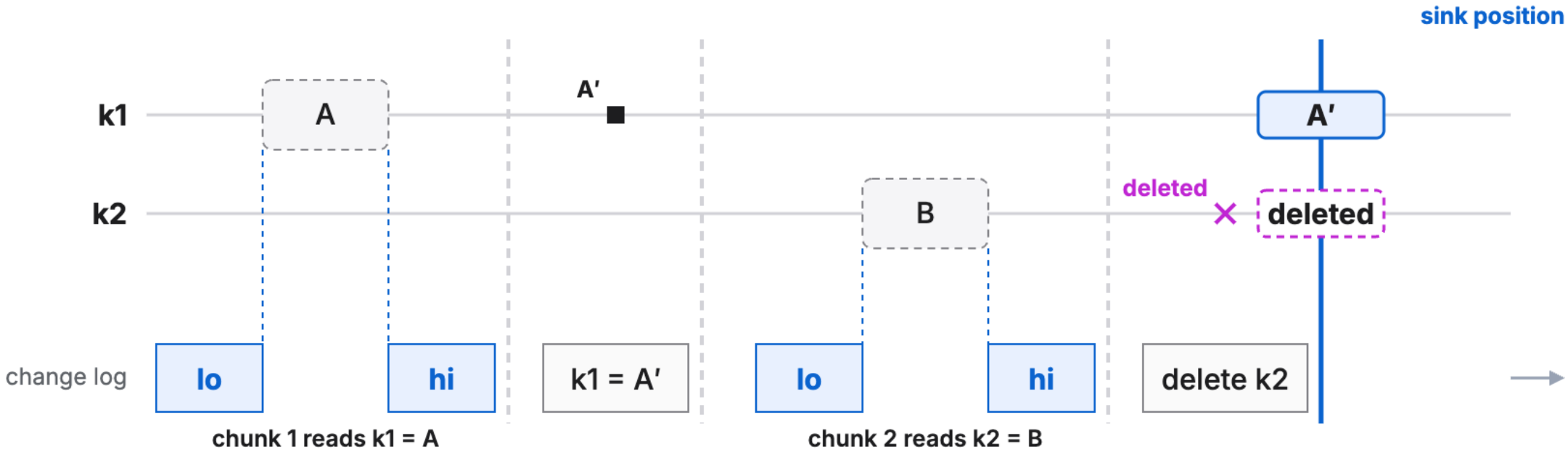


Every key now: k1 = A′ and k2 = B, exactly the table as of this position, although the two rows were read at different times.

No single read ever saw this snapshot, nothing was locked, and the connector kept nothing. That is snapshot equivalence.

SNAPSHOT EQUIVALENCE

Step 4: after a delete



k2 was deleted. Your sink applies the delete, and the line moves on: k1 = A′ and no k2, exactly as of this position.

A delete is just another change event. The row is gone at the source and gone in your sink.

FORMAL VERIFICATION WITH LLMS

Run the proofs yourself

```
git clone https://github.com/aandreakis/generalized-dblog-paper
cd generalized-dblog-paper/formal
python3 reproduce.py checksums
python3 reproduce.py isabelle --no-document
python3 reproduce.py lean
python3 reproduce.py tlc --jar /path/to/tla2tools.jar
```

Needs Isabelle2025-2, Lean 4.32.2, Python 3, Java 21 and, for the TLC runs, tla2tools.jar from the TLA+ v1.7.4 release, TLC 2.19.

The formal tree is byte-identical to the archived artifact: doi.org/10.5281/zenodo.22643866

AGENTS.md in the repository is written for LLM readers. Point your own model at it.